RESEARCH PAPER · PREPRINT

Smaller, Faster, Smarter

Loss-Gated Data Staging with Selective Token Replay for Compute-Efficient Small Language Model Training.

A survey of the four levers behind recent SLM efficiency gains, and CROP—a staged pretraining framework that couples document-level gating, selective token replay, a context ramp, and WSD compute recycling under one adaptive, FLOPs-accounted recipe.

Z.ai Research

Machine Learning Research
Z.ai

KEYWORDS
small language models · compute-efficient training · data curriculum · selective language modeling · knowledge distillation

Smaller, Faster, Smarter: Loss-Gated Data Staging with Selective Token Replay for Compute-Efficient Small Language Model Training

Z.ai Research

Preprint — November 2025

Abstract

Small language models (SLMs) are becoming the workhorse of on-device and cost-sensitive AI, yet their pretraining still spends compute the way frontier models do: every token in the corpus receives the same loss weight, every sequence is processed at full context length, and the training schedule is locked to a single compute budget. We first survey the four lever families that recent systems work has shown to matter most for SLM training efficiency: data curation, token-level selectivity, schedule design, and model compression. Motivated by this survey, we propose CROP (Curriculum-guided Reduction Of Pretraining), a staged training framework that couples (i) document-level gating driven by reference-model excess loss, (ii) token-level selective replay with a decaying mask share, (iii) a progressive context-length ramp, and (iv) a warmup–stable–decay schedule whose stable-phase checkpoint can be re-annealed for multiple budgets. We derive the FLOPs accounting of each component and project, for a representative 1.5B-parameter model trained on a 100B-token budget, a conservative total-compute reduction to approximately 88% of a uniform baseline, with a scenario reaching approximately 72% if the expected efficiency gain from selective replay materializes. All projections are analytic and clearly labeled; we specify a full experimental protocol—baselines, ablations, and evaluation suite—to test three concrete hypotheses. CROP is deliberately conservative: it introduces only two new hyper-parameters and reuses components that have each been validated in isolation, making it a practical near-term recipe rather than a speculative redesign.

Keywords: small language models; compute-efficient training; data curriculum; selective language modeling; learning-rate schedules; knowledge distillation

1 Introduction

Small language models—roughly, decoder-only transformers between 0.5B and 4B parameters—have moved from curiosities to deployment targets in their own right. Phi-3-mini (3.8B) reaches 69% on MMLU while being deployable on a phone [1], the Llama-3.2 family brings 1B and 3B models with 128K context to edge devices [2], and the Qwen2 and Gemma 2 releases show a dense cluster of competitive models at 1.5–3B [3, 4]. Because these models are retrained frequently—for new data mixes, new languages, new licenses, or new post-training pipelines—their *training* cost, not just their inference cost, is a first-order concern for any organization that ships them.

At the same time, the dominant training paradigm still treats all compute as equal. A standard run applies the next-token loss uniformly to every token, processes every sequence at the final context length from the first step, and commits the whole run to a single cosine schedule with a fixed endpoint. Scaling laws tell us this uniformity is wasteful in principle: compute-optimal training balances parameters and tokens [5, 6], and a growing body of work shows that *which* tokens are seen, *when* they are seen, and *how often* they are seen can shift the loss-versus-compute curve substantially at fixed parameter count [7–9].

This paper makes two contributions. First, we organize the recent, fast-moving literature on efficient SLM training into four lever families—data curation, token selectivity, schedule design,

and compression—and quantify what each has demonstrated on published systems (Section 3). Second, we propose CROP, a training framework that composes the four levers into a single recipe with an explicit FLOPs accounting (Sections 4 and 5). CROP is a proposal: every component it uses has been validated individually in prior systems work, but their composition, the adaptive stage-gating rule, and the compute-recycling loop are, to our knowledge, new. We state three falsifiable hypotheses and a complete experimental protocol (Section 6), and we discuss the risks and limitations of the approach honestly (Section 7).

2 Background: The Compute Economics of SLM Pretraining

2.1 Where the FLOPs go

For a dense transformer with N parameters trained on D tokens, the standard forward-plus-backward estimate for training compute is

$$C \approx 6\,N\,D, \tag{1}$$

which has been the workhorse accounting of the scaling-law literature [6]. The attention term adds a sequence-length-dependent component. With L_{layers} layers, model width d, and context length l, a useful refinement is

$$C(l) \approx 6D\,(N + L_{\text{layers}}\,l\,d), \tag{2}$$

so the attention share grows linearly with context length. For a representative 1.5B model (d=2048, L_{layers}=28), the attention term is roughly 8% of the parameter term at 2K context but about 15% at 4K; training the first half of a run at 2K therefore saves a few percent of total compute at no quality cost if quality is preserved. These two equations define the accounting we use throughout the paper, and every projection in Section 5 is derived from them by explicit arithmetic.

Equation (1) also explains why the SLM regime is peculiarly sensitive to data decisions. Because N is small, the $6ND$ budget of a 1.5B model over 100B tokens ($\approx 9 \times 10^{20}$ FLOPs) is five orders of magnitude below a frontier run; but the *fixed* costs—data preparation, tokenizer training, evaluation pipelines—do not shrink proportionally. A recipe that saves even 15–30% of training compute at equal quality therefore has an outsized practical effect on iteration speed.

2.2 The per-parameter capability gap

Published results show that parameter count is a poor predictor of SLM quality; the spread at fixed size is enormous. Figure 1(a) plots MMLU (5-shot) against parameter count for five recent open-weight SLMs using numbers from their own reports and model cards: at roughly 2.5–3.8B parameters, scores range from 51.3% (Gemma-2-2B) to 69.0% (Phi-3-mini). Figure 1(b) repeats the comparison against estimated training compute $6ND$ for models with published token counts: Phi-3-mini outperforms the compute-optimal Chinchilla-70B on MMLU while using an estimated $\sim$7.8$\times$ less training compute. The gap is attributable primarily to data—filtering, mixture, and synthesis—and secondarily to recipe choices such as annealing on high-quality data [1, 2, 9]. This is the central empirical motivation of our work: if data quality can buy such large multipliers, a training framework should *spend its FLOPs where the learning signal is*, and adapt that allocation as training proceeds.

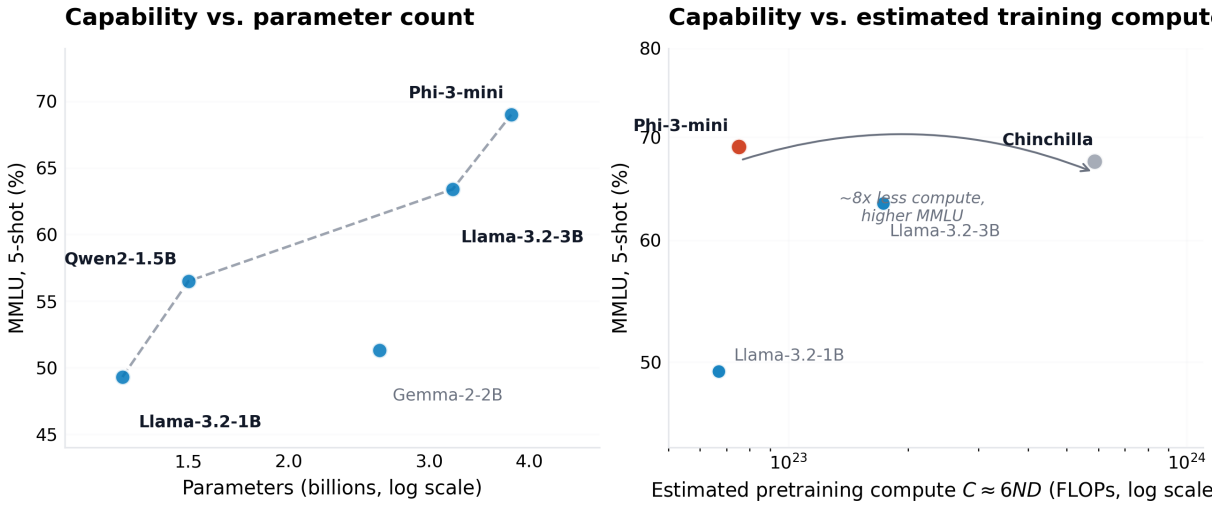


Figure 1: Published capability versus scale and compute for recent small language models. (a) MMLU (5-shot) against parameter count; numbers are taken from each model's own report or model card [1–4], and evaluation harnesses differ slightly across reports, so cross-model comparisons are indicative rather than exact. The dashed line is the Pareto frontier. (b) Estimated pretraining compute $C \approx 6ND$ against MMLU for models with published parameter and token counts (Llama-3.2 trained on up to 9T tokens [2]; Phi-3-mini on 3.3T [1]; Chinchilla on 1.4T [5]). Estimates are derived arithmetic, not measured FLOPs.

3 Survey: Four Levers for Faster SLM Training

We group the systems evidence into four families. The grouping matters because the levers compose: a data decision (what to train on) is orthogonal to a schedule decision (when to decay), and their benefits largely multiply rather than overlap.

3.1 Lever 1: Data curation and synthesis

The clearest lesson of the past three years is that data quality shifts the entire loss–compute curve. phi-1 (1.3B), trained on 6B filtered web tokens plus 1B synthetic "textbook" exercises for four days on eight A100s, reached 50.6% pass@1 on HumanEval—competitive with models an order of magnitude larger trained on unfiltered data [7]. TinyStories showed that with sufficiently clean synthetic data, models of only 1–10M parameters produce fluent coherent English [10]. At web scale, FineWeb-Edu—a 1.3T-token educational subset filtered from the 15T-token FineWeb corpus—dramatically improves knowledge- and reasoning-intensive benchmarks relative to its parent corpus [9]. DoReMi tunes domain mixtures with group-DRO and pretrains models that match baselines while using substantially less compute [11]. Phi-3-mini industrialized this stack: its "data-centric" recipe of heavily filtered and synthetic tokens is credited for most of its quality-per-parameter advantage [1]. Curation, however, is usually applied *once, before training*; the levers below make selectivity dynamic.

3.2 Lever 2: Token-level selectivity

Rho-1 directly challenges uniform loss weighting. Using a reference model to score tokens, its Selective Language Modeling trains only on tokens whose score exceeds a threshold; in continual pretraining on 15B OpenWebMath tokens it improves few-shot accuracy on math benchmarks by up to 30 absolute points, and its 1B/7B fine-tuned variants reach 40.6%/51.8% on MATH while matching DeepSeekMath with roughly 3% of the pretraining tokens [8]. The mechanism is intuitive: tokens the reference model already predicts well contribute near-zero gradient informa-

Table 1: Published small language models and the recipe elements most credited for their efficiency. MMLU numbers are 5-shot values from each model's own report or model card [1–4]; evaluation harnesses differ across reports, so cross-model gaps are indicative. Token counts are as disclosed; "n/a" denotes not disclosed at model level.

Model	Params	Tokens	MMLU	Key recipe element
Phi-3-mini [1]	3.8B	3.3T	69.0	Textbook-quality filtering and synthetic data
Llama-3.2-1B [2]	1.2B	$\leq$9T	49.3	Large multilingual mix; GQA; tied embeddings
Llama-3.2-3B [2]	3.2B	$\leq$9T	63.4	Same family recipe at 3B scale
Qwen2-1.5B [3]	1.5B	n/a	56.5	Multilingual mixture; GQA; QKV bias
Gemma-2-2B [4]	2.6B	n/a	51.3	Knowledge distillation from a larger teacher

tion but full compute cost. What Rho-1 leaves open—and what CROP exploits—is *scheduling* the selectivity: early in training almost every token is informative, while late in training the useful fraction concentrates in a shrinking subset.

3.3 Lever 3: Schedules, optimizers, and compute recycling

The warmup–stable–decay (WSD) schedule introduced with MiniCPM keeps the learning rate constant for most of the run and decays it late; MiniCPM shows WSD matches cosine-style training while leaving a stable-phase checkpoint that can later be annealed on different data for different budgets [12]. This enables *compute recycling*: one stable run fans out into many decayed endpoints, which is exactly how the Llama 3 team used annealing on small amounts of high-quality code and mathematics to squeeze out final-point gains [2]. On the optimizer axis, Muon—an orthogonalized-momentum optimizer—demonstrates approximately 2$\times$ computational efficiency versus AdamW at compute-optimal settings and was used to train the 3B/16B Moonlight MoE on 5.7T tokens [13]. Systems-level kernel work completes this family: FlashAttention reduces memory traffic of exact attention and materially raises hardware utilization during training [14].

3.4 Lever 4: Architecture, compression, and transfer

Small-model architecture itself is a lever. MobileLLM shows that, contra the "architecture does not matter" belief inherited from large-scale results, deep-thin designs, input–output embedding sharing, grouped-query attention, and immediate block-wise weight sharing lift sub-billion accuracy by 2.7–4.3% over prior state of the art at equal parameter count [15]; parameter sharing dates back at least to ALBERT [16]. Compression transfers capability into small models at a fraction of from-scratch cost: Minitron prunes a large teacher (width, depth, attention heads, MLP neurons) and recovers quality with knowledge distillation, improving MMLU by up to 16% over from-scratch training of the same size, with NVIDIA reporting training-cost savings up to 40$\times$ [17]. Reasoning-capability transfer follows the same pattern: DeepSeek-R1's distilled 1.5B–70B dense models, fine-tuned on roughly 800K teacher-generated reasoning traces, substantially outperform models RL-trained from scratch at those sizes [18, 19]. Table 1 summarizes the published SLM landscape that these levers have produced.

4 The CROP Framework

4.1 Overview

CROP (Curriculum-guided Reduction Of Pretraining) organizes a training run into an offline scoring pass and three training stages whose data mixture, context length, loss masking, and learning-rate phase are all coordinated (Figure 2). The design principle is to spend FLOPs where the reference-modeled learning signal is, and to spend it *increasingly selectively* as the model matures—a dynamic version of the static curricula of Section 3 [20]. Two properties distinguish CROP from a bag of tricks. First, every stage transition is *gated by a measured signal* (validation excess-loss slope), not by a fixed token count, so the recipe adapts to the corpus and model size. Second, the WSD backbone makes the whole run *re-usable*: the stable-phase checkpoint at the end of Stage B can be re-annealed repeatedly on different high-quality mixes, amortizing the bulk of the compute across many budget points, as sketched in the recycling branch of Figure 2.

4.2 Reference scoring with excess loss

Let $\ell(x_i;\theta)$ denote the next-token loss of the student on token x_i. Following the selective-pretraining literature [8], we score every token once with a small frozen reference model θ_{ref} (in this paper, a 0.3B model sharing the student's tokenizer):

$$s(x_i) = \ell(x_i;\, \theta_{\text{ref}}). \tag{3}$$

The intuition is that $s(x_i)$ measures how *surprising* a token is relative to competent language modeling: near-zero scores mark boilerplate that any reasonable model already predicts (and from which the student extracts little gradient), while high scores mark knowledge-dense or reasoning-critical content. Because scoring is a forward-only pass of a model $5\times$ smaller than the student, its cost is a small fraction of the training budget even when applied to a corpus larger than the tokens actually trained on; we quantify this in Section 5.

4.3 Stage A: document-level gating

Within a document d, let $\bar{s}_d$ be the mean token score. The Stage-A corpus keeps only documents whose aggregate signal clears a floor:

$$\mathcal{D}_A = \{\, d \in \mathcal{D} \;:\; \bar{s}_d \geq \theta_{\text{doc}} \,\}, \tag{4}$$

where θ_{doc} is set as a quantile of the document-level score distribution (we use the 30th percentile in projections). Document-level filtering is what actually *saves FLOPs*: dropped documents are never forwarded or back-propagated. Token-level masking alone cannot save compute, because masked tokens still flow through the network; this distinction drives the division of labor between the two gates. Stage A trains with the full next-token loss on $\mathcal{D}_A$ at 2K context, under the warmup-to-stable phase of the WSD schedule [12].

4.4 Stage B: selective token replay

Stage B replays the upper score tiers with a masked loss. With $\rho(t)$ the fraction of *easiest* tokens excluded at time t, and $q_\rho(t)$ the corresponding quantile of scores in the batch,

$$\mathcal{L}_\rho(\theta) = \frac{1}{\sum_i w_i} \sum_i w_i\, \ell(x_i;\theta), \qquad w_i = \mathbf{1}\big[s(x_i) > q_\rho(t)\big], \tag{5}$$

where $\rho(t)$ decays linearly from ρ_0 to ρ_1 across the stage. The decay encodes a curriculum in miniature: as the student improves, the frontier of genuinely informative tokens shifts upward, so the mask tightens and then relaxes toward the end of the stage to re-anchor on the full distribution. Context ramps linearly from 2K to 4K across Stage B, deferring the attention-term growth of Equation (2) until the model can exploit longer contexts.

Algorithm 1: CROP training procedure

Input: corpus $\mathcal{D}$, student size N, reference model θ_{ref}, total budget D^*, doc gate θ_{doc}, mask shares $\rho_0 > \rho_1$, plateau threshold γ, eval window W

Output: trained student θ^*

score $s(x_i) \leftarrow \ell(x_i; \theta_{\text{ref}})$ for all $x_i \in \mathcal{D}$; tier documents by $\bar{s}_d$;

$\mathcal{D}_A \leftarrow \{d : \bar{s}_d \geq \theta_{\text{doc}}\}$;

Stage A: train on $\mathcal{D}_A$ (ctx 2K, full loss, WSD warmup→stable);

Stage B: replay top tiers (ctx ramp 2K→4K, loss per Eq. (5));

  advance when Eq. (6) holds;

Stage C: decay LR on the top excess-loss slice; checkpoint θ^*;

return θ^*; optionally: reload the Stage-B checkpoint and re-anneal

  on a fresh slice for each additional budget;

4.5 Adaptive stage gating

CROP replaces fixed token counts with a measured transition rule. After every evaluation window of W steps we compute the slope of validation excess loss (the student's validation loss minus the reference model's validation loss on the same data). A stage transition fires when

$$\text{slope}_W < \gamma, \tag{6}$$

i.e., when marginal learning on the current mixture has stalled. This single rule, borrowed in spirit from early-stopping, is what makes the curriculum *adaptive*: a noisy corpus stalls early and advances to selectivity sooner; a rich corpus earns more stable-phase compute. Together with θ_{doc} (Section 4), γ is one of only two new hyper-parameters that CROP introduces beyond a standard WSD run.

4.6 Stage C: annealing and compute recycling

Stage C decays the learning rate over a short window on the highest-excess-loss, highest-quality slice—the same "anneal on the best data" practice validated by MiniCPM and the Llama 3 team [2, 12]. Because the WSD schedule places the decay entirely inside Stage C, the Stage-B stable checkpoint is a reusable asset: reloading it and re-annealing on a different slice yields an additional model for the cost of Stage C alone (15% of the budget in our projections). Algorithm 1 summarizes the full procedure.

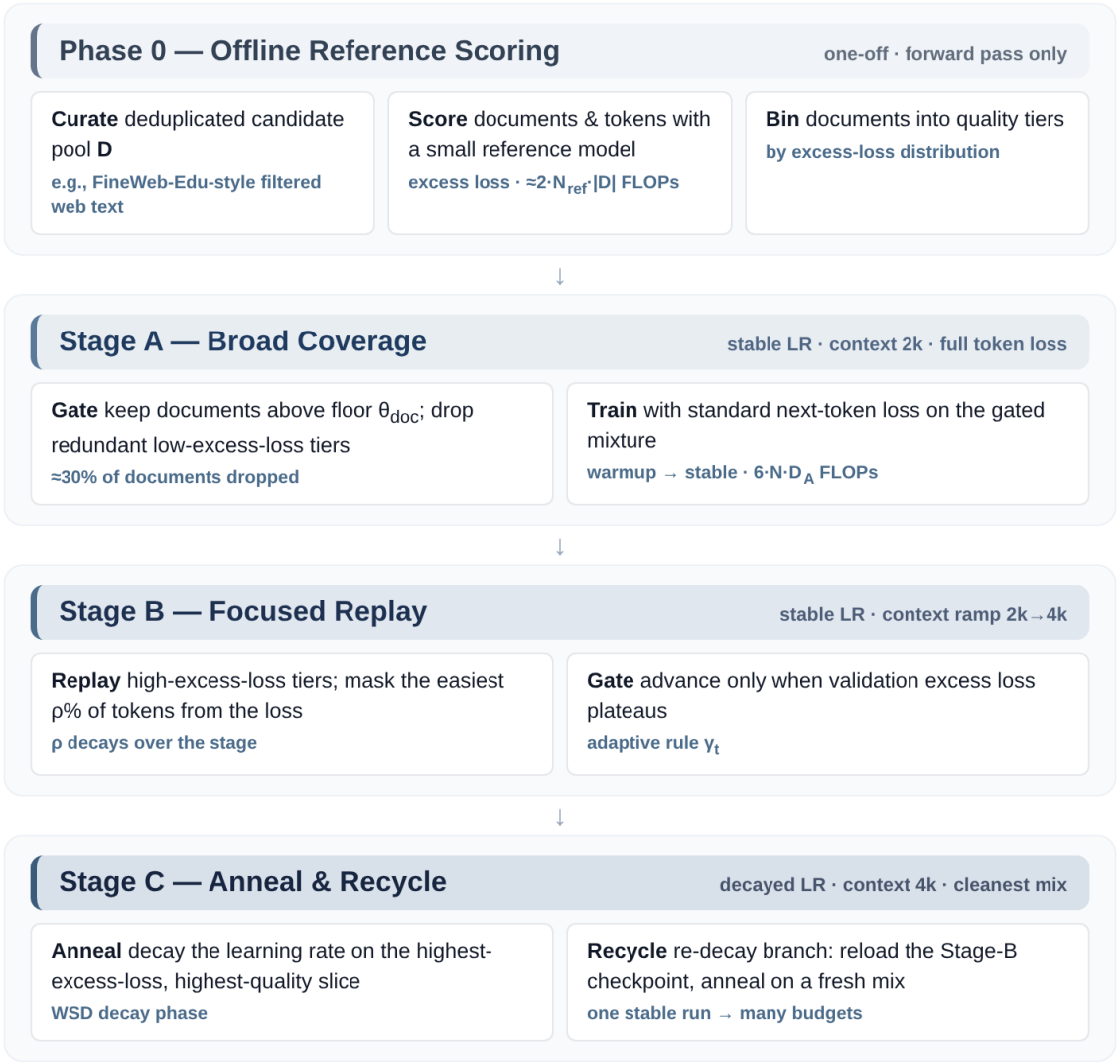


Figure 2: The CROP pipeline. A single offline pass of a small reference model scores every document and token (Phase 0); training then proceeds through three stages with increasingly selective data and loss (Stages A–C). The Stage-B stable checkpoint supports a re-anneal branch that serves additional compute budgets without repeating Stages A–B. Detailed per-stage configuration is given in the text and in the compute analysis of Table 2.

5 Compute Analysis: Projected FLOPs Budget

We now make the accounting explicit for a representative target: a 1.5B-parameter student (d=2048, L_{layers}=28), a 100B-token budget, stage shares of 55%/30%/15% for A/B/C, a 30% document drop in Stage A, a Stage-B context ramp whose midpoint is 3K, and a 0.3B reference model scoring a corpus 1.4× the token budget. Applying Equations (1) and (2): the uniform baseline (all 100B tokens at 4K context) costs $C_{\text{base}} \approx 1.04 \times 10^{21}$ FLOPs. Document gating removes 1.60×10^{20} FLOPs (−15.4%); the context ramp removes 6.9×10^{19} (−6.6%); selective token masking changes no FLOPs (masked tokens still propagate gradients) but, per the Rho-1 evidence, shifts the loss-versus-FLOPs curve downward [8]; and reference scoring adds 8.4×10^{19} (+8.1%). The conservative total is 9.14×10^{20} FLOPs—**88.0% of baseline** for the same token budget at equal-or-better expected quality.

If hypothesis H2 below holds (selective replay effectively shrinks the token budget needed for a given validation loss by 20%), the matched-loss scenario lands at 7.48×10^{20}—**72.1% of baseline**. We emphasize three caveats. First, these are *projections from a FLOPs model, not measurements*; their purpose is to size the prize and make the accounting falsifiable. Second, the projection deliberately excludes second-order effects that only help CROP: cheaper evaluation (smaller models), cheaper data pipeline reuse across runs, and the re-anneal branch, which serves each additional budget for roughly 15% of the original cost. Third, the numbers shift predictably with model scale: scoring overhead falls as the student grows relative to the fixed-size reference model, so the recipe is *more* attractive at 3–4B than at 1.5B.

Table 2: Projected FLOPs budget for a 1.5B student, 100B-token baseline budget, under the assumptions of Section 5. These are analytic projections from Equations (1) and (2)—**not measured results**. Token masking affects learning efficiency (the loss-versus-FLOPs curve), not raw FLOPs; its contribution is quantified only by the experiments of Section 6.

Component	Mechanism	FLOPs	vs. baseline
Uniform baseline	all 100B tokens, uniform 4K context, full loss	1.04×10^{21}	100%
Document gating	drop lowest-excess-loss 30% of Stage-A documents	-1.60×10^{20}	−15.4%
Context ramp	Stage A at 2K; Stage B ramped 2K→4K	-6.9×10^{19}	−6.6%
Token replay masking	mask easiest ρ% of tokens (no FLOP change)	0	0%
Reference scoring	0.3B forward pass over 1.4× corpus	$+8.4 \times 10^{19}$	+8.1%
CROP (conservative)	matched token budget	9.14×10^{20}	88.0%
CROP (H2 scenario)	matched validation loss, 20% effective-budget shift	7.48×10^{20}	72.1%

6 Experimental Protocol

6.1 Models, data, and baselines

We propose training at two scales—0.5B and 1.5B—on an open, reproducible data stack built from FineWeb-Edu [9], which provides both a broad pool and a quality-filtered subset, giving the reference scorer headroom to discriminate. Four baselines isolate each lever's contribution: (B1) a uniform WSD run on the full corpus (the compute baseline); (B2) B1 plus document gating only; (B3) B1 plus selective token masking only (the Rho-1-style ablation [8]); and (B4) B1 with cosine instead of WSD to test schedule sensitivity [12]. CROP is then compared against all four under two budgets: the same total FLOPs and the same wall-clock token count.

6.2 Hypotheses

The protocol tests three falsifiable hypotheses. **H1 (gating):** document-level gating with θ_{doc} at the 30th score percentile matches B1's validation loss using at least 12% fewer training

FLOPs. **H2 (replay):** Stage-B selective replay shifts the loss-versus-FLOPs curve by at least 15% relative to uniform replay of the same tokens (i.e., matched validation loss for 15–25% fewer effective tokens). **H3 (recycling):** a re-anneal from the Stage-B checkpoint recovers within 1 MMLU point of a full independent WSD run at the same added compute.

6.3 Evaluation and ablations

We report validation excess-loss curves against cumulative FLOPs (the primary efficiency view), plus the standard SLM suite—MMLU, HellaSwag, ARC-Challenge, GSM8K, and HumanEval—under published few-shot protocols [1, 3]. The ablation grid sweeps θ_{doc} over the 10th, 30th, and 50th score percentiles, ρ_0 over $\{0.2, 0.4\}$, γ over one decade, and the reference-model size N_{ref} over 0.15B, 0.3B, and 0.5B, to map the scoring cost–accuracy trade-off. Reference-model bias is probed by scoring with a mismatched-domain reference and measuring degradation; curriculum-order sensitivity is probed by reversing the tier order in Stage B. Every configuration runs with three seeds, and we report mean ± standard deviation.

7 Discussion and Limitations

CROP inherits the known risks of its ingredients, and adds composition risks of its own. Reference-model bias is the central one: if the scorer under-rates a domain (say, low-resource languages or code), the document gate will systematically starve that domain, and the harm compounds across stages. Our protocol probes this with mismatched-domain scorers, but a production recipe would want per-domain gate floors rather than a single global θ_{doc}. Second, curricula can be self-confirming: early selectivity narrows the distribution the student sees, which changes what the student finds surprising later. The decaying mask share $\rho(t)$ and the Stage-B re-anchoring are designed as counterweights, but the reversed-curriculum ablation is essential evidence. Third, the projections in Table 2 assume gating rates observed in Rho-1-style analyses transfer from continual pretraining to from-scratch training; if fresh students still need the easy tokens early, H1's savings shrink and the conservative scenario approaches the baseline. Fourth, our compute analysis uses the standard $6ND$ approximation and ignores pipeline, memory, and I/O effects that determine *wall-clock* throughput [14]; a systems evaluation on real clusters is required before the FLOPs story becomes a cost story. Finally, CROP is framed for dense SLMs; extending the gating idea to mixture-of-experts training, where per-token routing already interacts with selectivity, and combining it with optimizer-level gains [13] and compression pipelines [17] are natural next steps we leave open.

8 Conclusion

We surveyed four families of evidence—data curation, token selectivity, schedule design, and compression—that together explain most of the capability-per-FLOP progress in recent small language models, and we proposed CROP, a staged training framework that composes them under one adaptive, FLOPs-accounted recipe. CROP's contribution is not any single mechanism but the coupling: loss-gated document filtering that actually saves compute, token-level replay that improves the gradient signal per FLOP, a context ramp that defers attention costs, and a WSD backbone that turns one expensive run into many budget points through re-annealing. The projections are deliberately falsifiable, the protocol is fully specified, and every component has independent systems precedent. We offer CROP as a practical next step toward SLM training that is simultaneously faster, cheaper, and smarter per parameter.

References

[1] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell Hewett, Mojan Javaheripi, Piero Kauffmann, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

[2] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[3] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[4] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[5] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2203.15556.

[6] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[7] Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, Adil Salim, Shital Shah, Harkirat Singh Behl, Xin Wang, Sébastien Bubeck, Ronen Eldan, Adam Tauman Kalai, Yin Tat Lee, and Yuanzhi Li. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

[8] Zhenghao Lin, Zhibin Gou, Yeyun Gong, Xiao Liu, Yelong Shen, Ruochen Xu, Chen Lin, Yujiu Yang, Jian Jiao, Nan Duan, and Weizhu Chen. Rho-1: Not all tokens are what you need. *arXiv preprint arXiv:2404.07965*, 2024.

[9] Guilherme Penedo, Hynek Kydlíček, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. *arXiv preprint arXiv:2406.17557*, 2024.

[10] Ronen Eldan and Yuanzhi Li. Tinystories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023.

[11] Sang Michael Xie, Alessandro Santacroce, Kuan Zhai, and Tengyu Ma. Doremi: Optimizing data mixtures speeds up language model pretraining. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.16882.

[12] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Efficiently training language models with significantly fewer tokens. *arXiv preprint arXiv:2404.06395*, 2024.

[13] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.

[14] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2205.14135.

[15] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chenyu Lai, Yulu Shi, Sheng Zhu, Dingyou Zhang, et al. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14905.

[16] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1909.11942.

[17] Saurav Muralidharan, Sharath Sreenivas, Raviraj Joshi, Marcin Chochowski, Jan Kautz, Mohammad Shoeybi, and Bryan Catanzaro. Compact language models via pruning and knowledge distillation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.14679.

[18] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiong Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[19] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[20] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning (ICML)*, 2009.